DailyPlanet Technical Design Document

Ver 1.1

Contents

DailyPlanet Technical Design Document

1. Executive Summary

DailyPlanet is a news aggregation platform built using Next.js 16, React 19, and TypeScript. The system aggregates articles from multiple news providers, including The Guardian, NewsData.io, and Currents API, and presents them through a unified interface for browsing and search.

The application is designed around a provider-based architecture. Each provider implements a common interface, allowing the system to normalize different API responses into a shared article schema. An orchestration layer coordinates requests across providers, aggregates results, removes duplicate articles, and returns a chronologically sorted feed to the client.

The platform supports category-based browsing, keyword search, country filtering, date-range filtering, and infinite scrolling. To provide a consistent user experience despite differing provider pagination mechanisms, the system employs a composite cursor pagination model that combines provider-specific pagination state into a single encoded token.

Performance and reliability are improved through parallel provider requests, server-side response caching, Redis-backed rate limiting, and fault-tolerant request handling. The architecture is designed to be extensible, allowing additional news providers to be integrated with minimal changes to the core system.

This document describes the architecture, component interactions, design decisions, data flow, and scalability considerations of the DailyPlanet platform.

2. High-Level Architecture

DailyPlanet uses a layered architecture to aggregate news from multiple external providers through a unified interface. User requests are handled by Next.js API routes, which delegate processing to an orchestration layer responsible for provider selection and request execution.

Provider responses are normalized into a common article schema before being aggregated, deduplicated, sorted, and returned to the client. This design isolates provider-specific logic while providing a consistent experience for search, filtering, and pagination

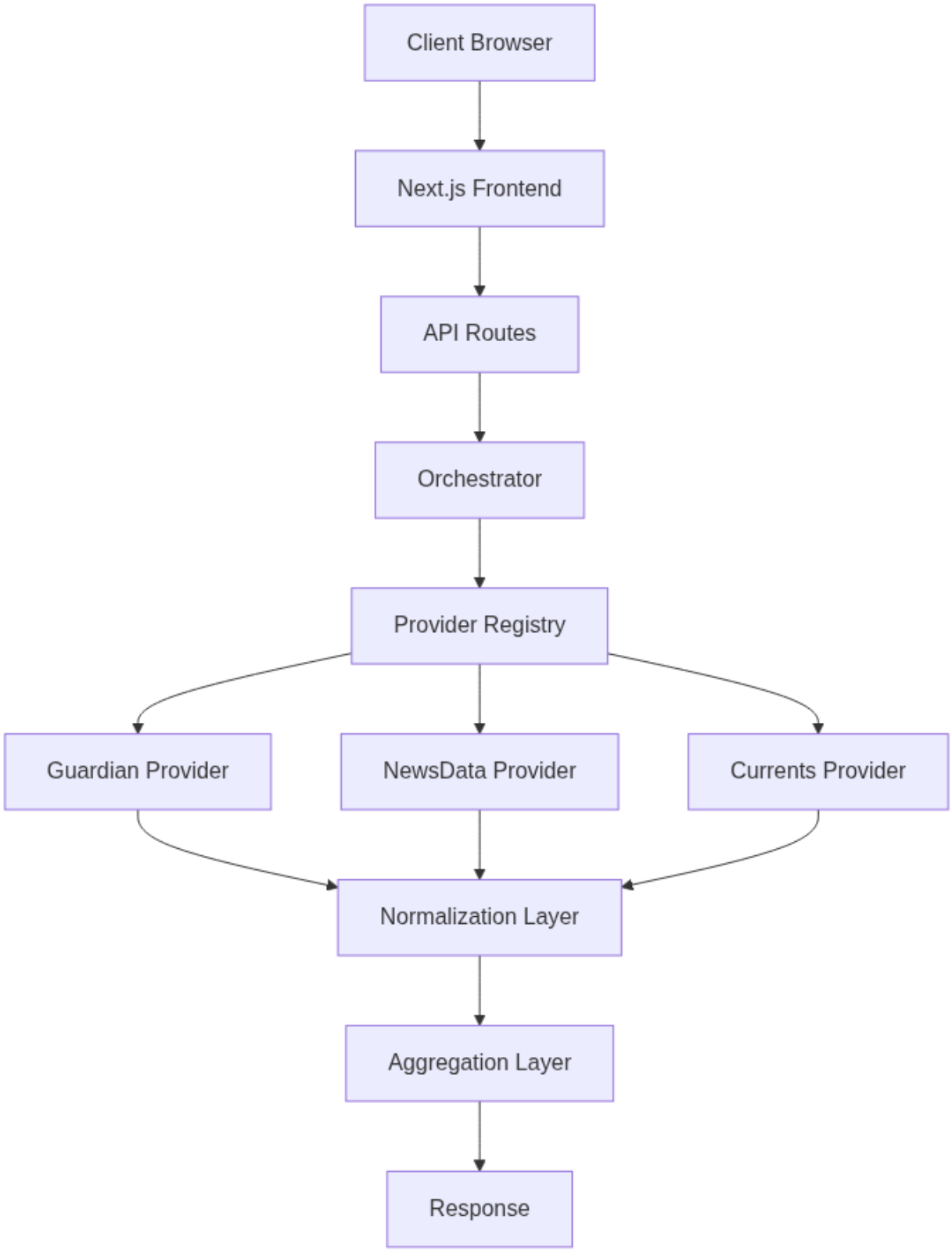

Figure 1: High-Level Architecture

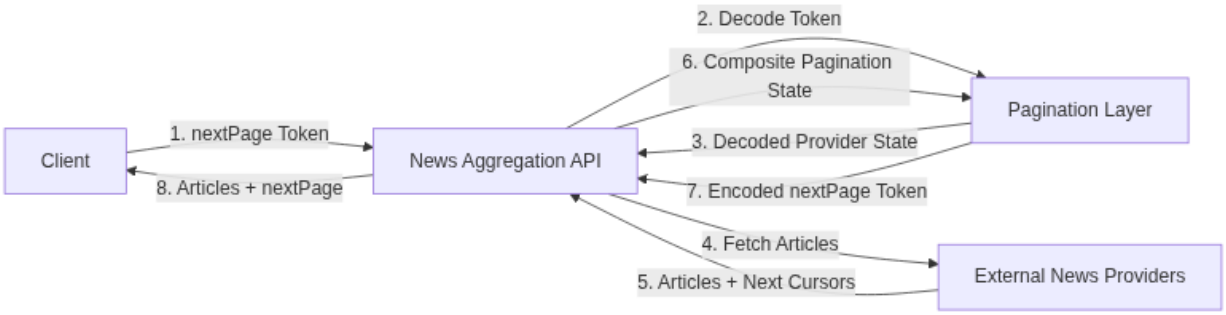

Figure 2: cursor flow diagram

3. Technology Stack

Technology	Version	Purpose
Next.js	16.2.6	Frontend framework and API routes
React	19.2.4	User interface development
TypeScript	5.9.3	Type safety and maintainability
Upstash Redis	1.38.0	Distributed rate limiting
Guardian API	N/A	News provider
NewsData.io	N/A	News provider
Currents API	N/A	News provider
Vercel	N/A	Application hosting and deployment
Playwright	1.6.0	API and E2E testing
Tailwind CSS	4.3.0	Styling

3.1 Technology Selection Rationale

Several technologies were selected based on their suitability for a scalable, maintainable, and serverless news aggregation platform.

- **Next.js 16** was chosen because it provides both frontend rendering and backend API route capabilities within a single framework. This simplifies development by enabling the entire application stack to be implemented using a unified technology platform while also streamlining deployment and maintenance.
- **React 19** enables the development of reusable and modular user interface components. Its component-based architecture improves maintainability, promotes code reuse, and supports the creation of responsive user experiences.
- **TypeScript 5.9.3** was selected to improve code quality and maintainability through static type checking. Strong typing reduces runtime errors, improves tooling support, and makes the codebase easier to understand and extend.
- **Guardian API**, **NewsData.io**, and **Currents API** were selected to provide diverse and complementary news sources. Integrating multiple providers improves article coverage across categories and regions while reducing dependence on any single external service.

- **Vercel** was chosen as the deployment platform due to its native support for Next.js applications, automated deployment workflows, serverless execution model, and globally distributed infrastructure. These features simplify application hosting while supporting scalability and high availability.
- **Playwright** was selected for testing because it supports both API testing and end-to-end browser testing within a single framework. This enables automated verification of application functionality and improves confidence in system reliability.
- **Tailwind CSS 4.3.0** was chosen to accelerate frontend development through utility-first styling. Its approach encourages consistent design patterns while reducing the amount of custom CSS required.
- **Upstash Redis** was selected for distributed rate limiting because it provides a serverless Redis solution that integrates well with Vercel deployments. Alternatives such as in-memory rate limiting were unsuitable because rate-limit counters would not be shared across multiple serverless instances, while self-hosted or traditionally managed Redis deployments would introduce additional infrastructure management overhead. Upstash provides persistent, shared storage for rate-limit data without requiring dedicated infrastructure.

4. System Components

This section describes the major software components that make up the DailyPlanet platform and their responsibilities.

4.1 Frontend Components

The frontend is built using Next.js and React and is responsible for displaying aggregated news content and managing user interactions.

Component	Responsibility
NewsFeed	Displays articles and manages infinite scrolling
NewsCard	Renders individual article content and metadata
GlobalSearchBar	Handles keyword-based searching
CountryFilter	Applies country-based filtering
CatTabs	Manages category selection and navigation

4.2 API Layer

The API layer exposes server-side endpoints used by the frontend.

`/api/news` Responsible for:

- Retrieving aggregated headlines
- Applying filters
- Managing pagination

- Enforcing rate limits

`/api/search` Responsible for:

- Executing cross-provider searches
- Returning normalized search results
- Managing pagination

4.3 Backend Components

Component	Location	Responsibility
Orchestrator	src/lib/news/orchestrator.ts	Coordinates provider execution
Registry	src/lib/news/registry.ts	Selects compatible providers
Normalizer	src/lib/normalize.ts	Standardizes provider responses
Aggregator	src/lib/aggregate.ts	Merges and deduplicates articles
Pagination	src/lib/news/pagination.ts	Generates composite cursors
Rate Limiter	src/lib/rateLimit.ts	Enforces API limits

5. Request Flow

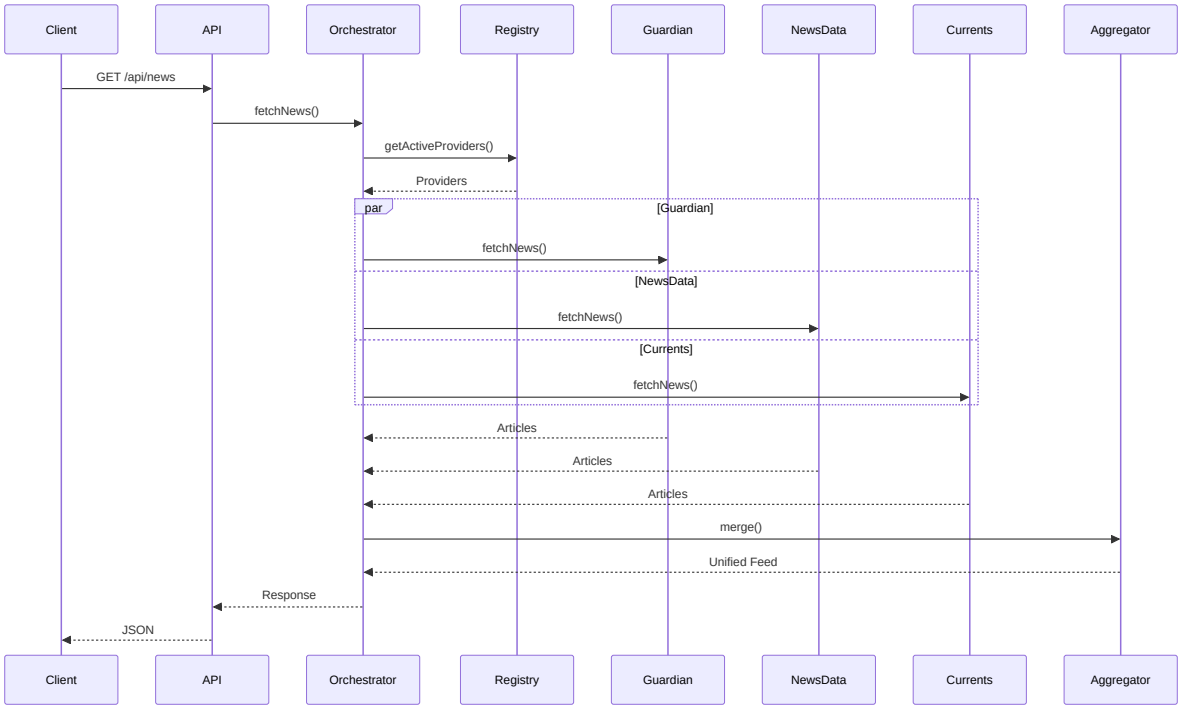

Figure 3: Request Flow

5.1 News Feed Flow

1. The client sends a request to `/api/news` with optional filtering and pagination parameters.
2. The API layer validates the request and applies rate limiting.
3. The orchestration layer selects compatible providers through the provider registry.
4. Active providers are queried concurrently.
5. Responses are normalized, aggregated, deduplicated, and sorted.
6. A composite pagination cursor is generated.
7. The aggregated response is returned to the client.

5.2 Search Flow

The search workflow follows the same execution pipeline as the news feed workflow, but includes a user-provided search query and executes provider search operations instead of headline retrieval.

6. Provider Architecture

DailyPlanet uses a provider-based architecture to abstract interactions with external news services. Each news source is implemented as a provider that conforms to a common interface, allowing the rest of the system to interact with all providers in a consistent manner regardless of their underlying API implementations.

This approach isolates provider-specific logic from the orchestration and aggregation layers, simplifying maintenance and enabling new providers to be integrated with minimal changes to the core system.

6.1 Provider Interface

All providers implement a shared interface that defines the operations required by the orchestration layer.

The interface standardizes:

- News retrieval
- Search operations
- Pagination handling
- Provider capability metadata
- Response normalization

By enforcing a common contract, the orchestration layer can interact with all providers without requiring provider-specific logic.

6.2 Provider Registry

The provider registry maintains a list of all available providers and is responsible for provider selection.

Before a request is executed, the registry evaluates provider capabilities and excludes providers that cannot satisfy the requested filters.

Current provider capabilities include:

- Country filtering support
- Date-range filtering support

This ensures that only compatible providers participate in request execution.

6.3 Provider Implementations

The system currently supports three providers:

1. Guardian
2. NewsData.io
3. Currents

Each provider is responsible for:

- Constructing provider-specific API requests
- Mapping categories to provider-specific values
- Handling pagination
- Normalizing responses into the common article schema

6.4 Provider Execution

Once active providers have been selected, the orchestration layer executes provider requests concurrently using JavaScript's `Promise.all()` mechanism.

```
const results = await Promise.all(
  providers.map(async (provider) => {
    try {
      return await provider.fetch(request)
    } catch {
      return {
        articles: [],
        nextCursor: undefined
      }
    }
  })
)
```

6.5 Extensibility

The provider architecture was designed to support future expansion.

To integrate a new news source, a developer must:

1. Implement the provider interface.
2. Register the provider in the provider registry.
3. Configure category mappings and provider capabilities.

No modifications to the orchestration or aggregation layers are required, allowing the platform to scale horizontally as additional providers are introduced.

7. Filtering System

DailyPlanet supports category, country, and date-range filtering across multiple news providers. Since providers expose different filtering capabilities, the system uses capability-based provider selection to ensure only compatible providers participate in request execution.

Before a request is processed, the provider registry evaluates the requested filters and excludes providers that cannot satisfy them. This prevents unsupported filters from producing inconsistent or inaccurate results.

7.1 Category Filtering

Category filtering is supported by all providers.

User-facing categories are mapped to provider-specific category values through a centralized category mapping layer. This abstraction allows a single category selection to be translated into the appropriate format required by each external API.

7.2 Country Filtering

Country filtering is supported by NewsData and Currents providers.

When a country filter is specified, providers that do not support country-based filtering are automatically excluded from execution.

This ensures that all returned articles satisfy the requested geographic constraints.

7.3 Date Filtering

Date-range filtering is supported by Guardian and Currents providers.

When a date range is supplied, the provider registry excludes providers that cannot perform date-based filtering.

This allows the system to return results that accurately reflect the requested time window without requiring additional client-side filtering.

7.4 Capability-Based Provider Selection

Provider selection is performed dynamically using capability metadata exposed by each provider.

```
if (opts.country && !provider.supportsCountryFilter)
  return false

if (opts.startDate && opts.endDate && !provider.supportsDateFilter)
  return false
```

The registry evaluates provider capabilities before request execution and automatically excludes providers that cannot satisfy the requested filters.

Provider	Category Filter	Country Filter	Date Filter
Guardian	✓	x	✓
NewsData	✓	✓	x
Currents	✓	✓	✓

This approach allows filtering behavior to remain consistent while supporting providers with different feature sets.

7.5 All-Provider Exclusion

If a filter combination excludes all enabled providers, the system does not return a special error. Instead, the orchestrator completes successfully and returns an empty article list.

Because provider selection is performed before any provider fetch occurs, this edge case is handled as an empty result set rather than a failure. The response still follows the normal API shape, but no provider-specific results are included.

8. Pagination Design

DailyPlanet aggregates content from multiple news providers, each of which exposes a different pagination mechanism. While Guardian and Currents use page-based pagination, NewsData uses cursor-based pagination.

This creates a challenge when presenting a single unified feed to the client, as the application must maintain pagination state for multiple providers simultaneously.

8.1 Problem Statement

A traditional pagination model assumes that all data originates from a single source with a single pagination strategy.

In DailyPlanet, each provider maintains its own pagination state:

- Guardian uses page numbers
- Currents uses page numbers
- NewsData uses cursor tokens

As a result, the client cannot directly interact with provider-specific pagination mechanisms.

8.2 Composite Cursor Model

To provide a consistent pagination experience, DailyPlanet uses a composite cursor model.

The pagination state of all active providers is combined into a single object:

```
{
  "guardianPage": 2,
  "newsDataCursor": "abc123",
  "currentsPage": 3
}
```

This object represents the complete pagination state required to retrieve the next batch of results.

8.3 Cursor Encoding

The composite pagination state is encoded and returned to the client as a single pagination token.

The client treats this token as an opaque value and includes it in subsequent requests when requesting additional results.

This approach hides provider-specific implementation details from the client while maintaining support for heterogeneous pagination strategies.

8.4 Pagination Workflow

1. Providers return articles and updated pagination state.
2. The orchestration layer combines provider pagination information.
3. The pagination layer generates a composite cursor.
4. The cursor is encoded and returned as the `nextPage` value.
5. The client supplies the cursor in subsequent requests.
6. The pagination layer decodes the cursor and restores provider-specific state.

8.5 Page Size Behavior

DailyPlanet does not enforce a single global maximum page size for the composite feed. The number of articles returned per request depends on the active providers and their individual batching behavior.

- `Currents` is explicitly configured with `page_size=10`.
- `Guardian` and `NewsData` use their provider defaults because no page size override is set in the current implementation.
- The final article count is the result of aggregating provider batches and de-duplicating overlapping content, so the actual number returned may be higher or lower than any one provider's page size.

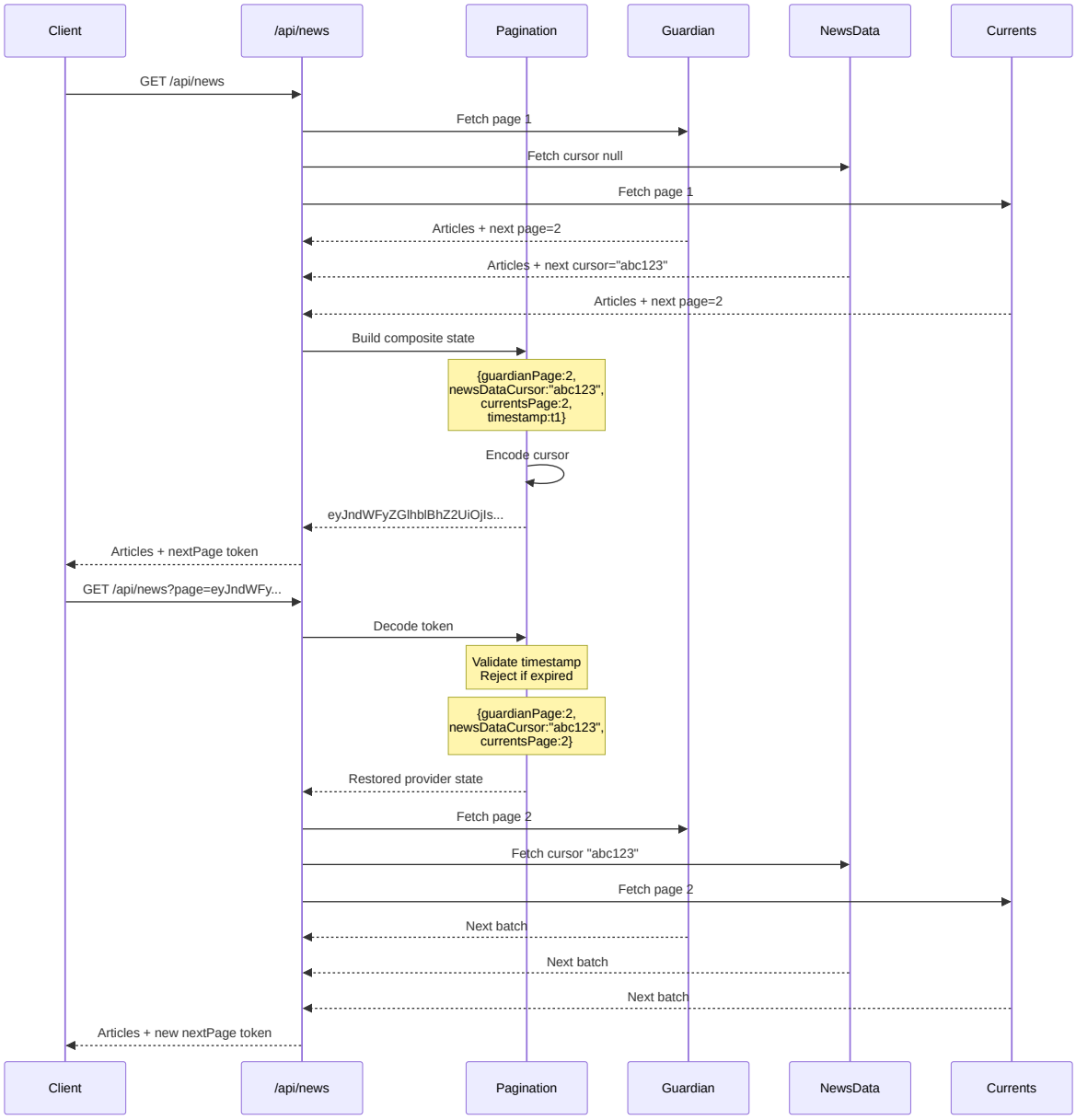

Figure 4: cursor request flow diagram

8.6 Cursor Expiration and Stale Cursors

The composite pagination token is a base64-encoded representation of provider-specific pagination state plus an encoded timestamp.

- Each cursor is generated with `encodePagination()` and includes `Date.now()`.
- `decodePagination()` validates the cursor age against a freshness window (currently 5 minutes).
- If the cursor is older than the configured window, it is treated as expired and the orchestrator resets pagination to the first page for all providers.
- This expired cursor behavior is handled silently: the API still returns a successful response with fresh results and a new composite cursor.
- Clients should avoid reusing old cursors for long-lived sessions and should prefer the latest `nextPage` token returned by the API.

8.7 Benefits

The composite cursor model provides several advantages:

- Unified pagination across multiple providers
- Provider-specific pagination complexity remains hidden from clients
- Support for heterogeneous pagination mechanisms
- Simplified frontend implementation
- Extensibility for future providers

9. Error Handling & Fault Tolerance

DailyPlanet incorporates multiple error handling mechanisms to ensure reliability when interacting with external services and processing client requests.

9.1 Request Validation

Incoming requests are validated before processing begins.

Validation includes:

- Required parameter checks
- Date range validation
- Category validation
- Pagination cursor validation

Invalid requests are rejected with an appropriate error response before any provider requests are executed.

9.2 Provider Failures

The platform depends on multiple third-party news providers, each of which may experience downtime, rate limits, or unexpected API changes.

Provider requests are isolated from one another, allowing the system to continue operating even if an individual provider fails.

If a provider encounters an error, the failure is logged and remaining providers continue processing the request.

This approach prevents a single provider outage from making the entire service unavailable.

9.3 Environment Validation

During application startup, the provider registry validates that all required API credentials are present.

```
const missingKeys: string[] = []

if (!process.env.GUARDIANAPI_KEY)
  missingKeys.push("GUARDIANAPI_KEY")

if (!process.env.NEWSDATA_API_KEY)
  missingKeys.push("NEWSDATA_API_KEY")

if (!process.env.CURRENTNEWS_API_KEY)
  missingKeys.push("CURRENTNEWS_API_KEY")
```

If required configuration values are missing, the application fails fast and reports the missing environment variables.

9.4 Rate Limiting

To prevent abuse and protect external API quotas, server-side rate limits are enforced using Upstash Redis.

Requests exceeding configured limits receive an error response and are prevented from executing provider requests.

This protects both infrastructure resources and third-party API quotas.

9.5 Fault Tolerance Strategy

The system is designed to degrade gracefully when failures occur.

Key fault-tolerance measures include:

- Provider isolation
- Independent provider execution
- Partial result delivery
- Configuration validation
- Rate limiting

These mechanisms ensure that the platform remains operational even when external dependencies become unavailable.

10. Performance Optimizations

DailyPlanet incorporates several optimizations to reduce response times, minimize external API usage, and improve scalability.

10.1 Parallel Provider Execution

News providers are queried concurrently rather than sequentially.

By executing provider requests in parallel, the total response time is determined by the slowest provider rather than the combined latency of all providers.

This significantly reduces the time required to aggregate content from multiple sources.

10.2 Server-Side Caching

External API requests are cached using Next.js fetch revalidation.

Cached responses can be served without contacting the underlying provider, reducing latency and decreasing the number of external API requests.

Caching also helps preserve limited API quotas provided by third-party services.

10.3 Infinite Scrolling

The frontend implements infinite scrolling to load articles incrementally.

Instead of retrieving a large number of articles during the initial request, content is loaded in smaller batches as the user navigates through the feed.

This reduces initial page load times and improves perceived responsiveness.

10.4 Aggregation and Deduplication

Provider responses are normalized and aggregated on the server before being returned to the client.

Duplicate articles are removed during aggregation, reducing unnecessary data transfer and preventing repeated content from appearing in the feed.

10.5 Rate Limiting

Server-side rate limiting is enforced using Upstash Redis.

Rate limiting protects external API quotas and prevents excessive request volumes from impacting application performance.

By controlling traffic at the API layer, the system can maintain predictable resource utilization under varying load conditions.

11. API Specification

DailyPlanet exposes two primary API endpoints that provide access to aggregated news content and search functionality.

11.1 News Endpoint

Route

```
GET /api/news
```

Response Status

- `200 OK` - Success, articles returned
- `400 Bad Request` - Invalid parameters
- `429 Too Many Requests` - Rate limit exceeded
- `500 Internal Server Error` - Server-side failure

Purpose

Returns aggregated news articles from all compatible providers.

Supported Parameters

Parameter	Description
category	Filter articles by category
country	Filter articles by country
startDate	Start of date range
endDate	End of date range
page	Composite pagination cursor

Response Structure

```
{
  "success": true,
  "articles": [...],
  "nextPage": "encodedCursor"
}
```

11.2 Search Endpoint

Route

```
GET /api/search
```

Response Status

- `200 OK` - Success, search results returned
- `400 Bad Request` - Invalid parameters

- `429 Too Many Requests` - Rate limit exceeded
- `500 Internal Server Error` - Server-side failure

Purpose

Performs keyword-based searches across all compatible providers.

Supported Parameters

Parameter	Description
query	Search term
category	Filter results by category
country	Filter results by country
startDate	Start of date range
endDate	End of date range
page	Composite pagination cursor

Response Structure

```
{
  "success": true,
  "articles": [...],
  "nextPage": "encodedCursor"
}
```

11.3 Unified Article Schema

All provider responses are normalized into a common schema before being returned to clients.

```
{
  title: string
  url: string
  category: string[]
  source: string
  publishedAt: string
  content: string
  byline?: string
  thumbnail?: string
}
```

The aggregation layer normalizes responses from multiple providers into a unified article schema. Some fields are optional because they depend on the metadata supplied by the source provider.

Field	Guardian	NewsData	Currents
title	Yes	Yes	Yes
url	Yes	Yes	Yes

Field	Guardian	NewsData	Currents
category	Yes	Yes**	Yes**
source	Yes	Yes	Yes
publishedAt	Yes	Yes	Yes
content	Yes*	Yes*	Yes*
byline	Yes*	Yes*	Yes*
thumbnail	Yes*	Yes*	Yes*

* Availability depends on the metadata supplied by the provider for a specific article.

** If no category is supplied by the provider, the normalization layer assigns the default category `general`.

This abstraction allows client applications to consume news content without requiring knowledge of provider-specific response formats.

11.4 Pagination

Both endpoints support cursor-based pagination through the `page` parameter.

The cursor contains encoded provider-specific pagination state and is generated by the pagination layer.

Clients should treat the cursor as an opaque value and pass it unchanged in subsequent requests.

11.5 Error Responses

Error responses follow a consistent structure:

```
{
  "success": false,
  "error": "Error message"
}
```

The following HTTP status codes are used:

Status Code	Scenario
400	Invalid request parameters (e.g., invalid category, invalid date range)
429	Rate limit exceeded for the requesting IP address
500	Internal server error (e.g., cursor decoding failure, provider request failure)

Common error cases:

- **400 Bad Request**: Invalid request parameters

  - Invalid category filter
  - Invalid date range (end date before start date, range exceeds 15 days)
  - Invalid pagination cursor
  - Missing required parameters
- **429 Too Many Requests**: Rate limit violations
  - Excessive requests from a single IP address within a time window
- **500 Internal Server Error**: Server-side failures
  - Provider API failures (after graceful handling)
  - Cursor decoding failures
  - Unexpected service errors

12. Security Considerations

12.1 API Key Management

All external API keys (Guardian, NewsData.io, Currents API) are stored as environment variables and are not hardcoded in the codebase.

- API keys are loaded from `.env` / deployment environment configuration
- Keys are never exposed to the client-side bundle
- Access is restricted to server-side API routes only

In production, these environment variables are managed through the deployment platform (e.g., Vercel environment settings), ensuring they are not stored in source control.

12.2 Key Rotation

API keys can be rotated by updating the environment variables in the deployment configuration without requiring code changes.

- Old keys are invalidated at the provider level
- New keys take effect on the next deployment or server restart
- No runtime dependency on fixed credentials

12.3 Input Sanitization

User input, particularly the `query` parameter used for search, is treated as untrusted input.

- Query strings are trimmed before use
- Empty or whitespace-only queries are handled as “headlines mode” (no search performed)
- Inputs are passed to providers via server-side requests only (no direct client exposure)
- URL encoding is handled automatically by `URLSearchParams`, preventing injection attacks on external APIs

Additionally: - Category values are validated against a predefined whitelist (`CATEGORY_MAP`) - Invalid categories are rejected with a `400 Bad Request` response - Date ranges are validated: end date must be after start date, and range cannot exceed 15 days - Query length is limited to 200 characters to prevent resource exhaustion

12.4 External API Safety

All requests to external providers are made from the server layer only, preventing direct client access to third-party APIs and reducing exposure of API keys.

- Provider API keys are never sent to the client
- All API calls are performed server-to-server
- Rate limiting is applied at the server layer before provider requests are made
- Failed provider requests are handled gracefully without exposing internal error details to clients

12.5 Error Handling & Information Disclosure

Error responses are designed to avoid exposing sensitive system information.

- Generic error messages are returned for server-side failures (500 errors)
- Specific validation error messages are returned for client errors (400 errors)
- Cursor decoding failures are caught and converted to validation errors
- Provider-specific error details are logged server-side but not exposed to clients

13. Testing Strategy

DailyPlanet is designed to support testing at multiple levels, including unit testing, integration testing, and end-to-end testing. This layered approach ensures that individual components, system interactions, and user workflows can be validated independently.

13.1 Unit Testing

Unit tests focus on validating isolated business logic without requiring external dependencies.

Key areas for unit testing include:

- Article normalization
- Category mapping
- Aggregation and deduplication
- Cursor encoding and decoding
- Provider selection logic

These tests ensure that core functionality behaves correctly under a variety of inputs and edge cases.

13.2 Integration Testing

Integration tests verify interactions between internal components and API routes.

Key integration test scenarios include:

- News retrieval through `/api/news`
- Search functionality through `/api/search`
- Pagination handling
- Filter application

- Rate limiting behavior

Integration testing ensures that requests are processed correctly throughout the application pipeline.

13.3 External API Mocking

Third-party news providers should be mocked during automated testing to avoid dependency on external services.

Mocking provides several benefits:

- Consistent test results
- Faster execution
- Reduced API quota consumption
- Isolation from provider downtime

Provider responses can be simulated using representative sample payloads obtained from Guardian, NewsData, and Currents APIs.

13.4 End-to-End Testing

End-to-end testing validates complete user workflows through the frontend interface.

Example scenarios include:

- Browsing news articles
- Performing searches
- Applying category filters
- Applying country filters
- Applying date-range filters
- Loading additional content through infinite scrolling

These tests verify that the application behaves correctly from a user's perspective.

13.5 Test Coverage Goals

The primary testing focus should be placed on the system's core business logic and integration points.

High-priority areas include:

- Provider orchestration
- Capability-based provider selection
- Pagination state management
- Aggregation and deduplication
- API endpoint behavior

Testing these components provides confidence in the correctness and reliability of the platform.

14. Scalability Considerations

DailyPlanet was designed with scalability and maintainability as primary architectural goals. The system's modular structure allows individual components to evolve independently as requirements grow.

14.1 Provider Scalability

The provider-based architecture allows additional news sources to be integrated without modifying the orchestration or aggregation layers.

To add a new provider, a developer must:

1. Implement the provider interface.
2. Register the provider in the provider registry.
3. Configure category mappings and capability metadata.

This approach enables the platform to scale horizontally as additional content sources are introduced.

14.2 Feature Scalability

The capability-based provider selection mechanism allows new filtering features to be introduced incrementally.

Providers can advertise support for new capabilities without requiring changes to existing provider implementations.

This reduces coupling between providers and core application logic.

14.3 Traffic Scalability

The application is built on Next.js server-side infrastructure and incorporates caching and rate limiting to reduce load on both internal services and external providers.

As traffic increases, additional caching strategies and distributed deployments can be introduced without significant architectural changes.

14.4 Data Source Scalability

Because all provider responses are normalized into a common article schema, the aggregation layer remains independent of provider-specific response formats.

This ensures that increasing the number of providers does not significantly increase system complexity.

14.5 Future Scaling Opportunities

Potential future enhancements include:

- Additional news providers
- Distributed caching layers

- Search indexing for faster search performance
- Provider health monitoring
- Personalized news recommendations
- Analytics dashboards

15. Future Improvements

Several enhancements can be made to improve functionality, scalability, and user experience.

- Integration of additional news providers
- Full-text search indexing
- Personalized article recommendations
- Provider health monitoring
- Analytics and reporting dashboards
- Advanced caching strategies
- User accounts and saved article functionality

These improvements can be implemented without significant architectural changes due to the modular provider-based design of the platform.